

SkillClaw

Collective Skill Evolution for Multi-User LLM Agent Ecosystems

Skills continuously evolve by aggregating cross-user interaction trajectories and applying agentic, evidence-driven skill updates.

Agent Skills Remain Static After Deployment

Users Rediscover Solutions Independently

STATIC SKILLS (Today)

- Skills installed manually from a hub
- Never updated from runtime experience
- Failures & solutions silently discarded
- Each user rediscovers the same fix
- Knowledge never accumulates at system level
- Memory methods: tied to specific instances, hard to generalize
- Skill libraries: static, not updated by usage

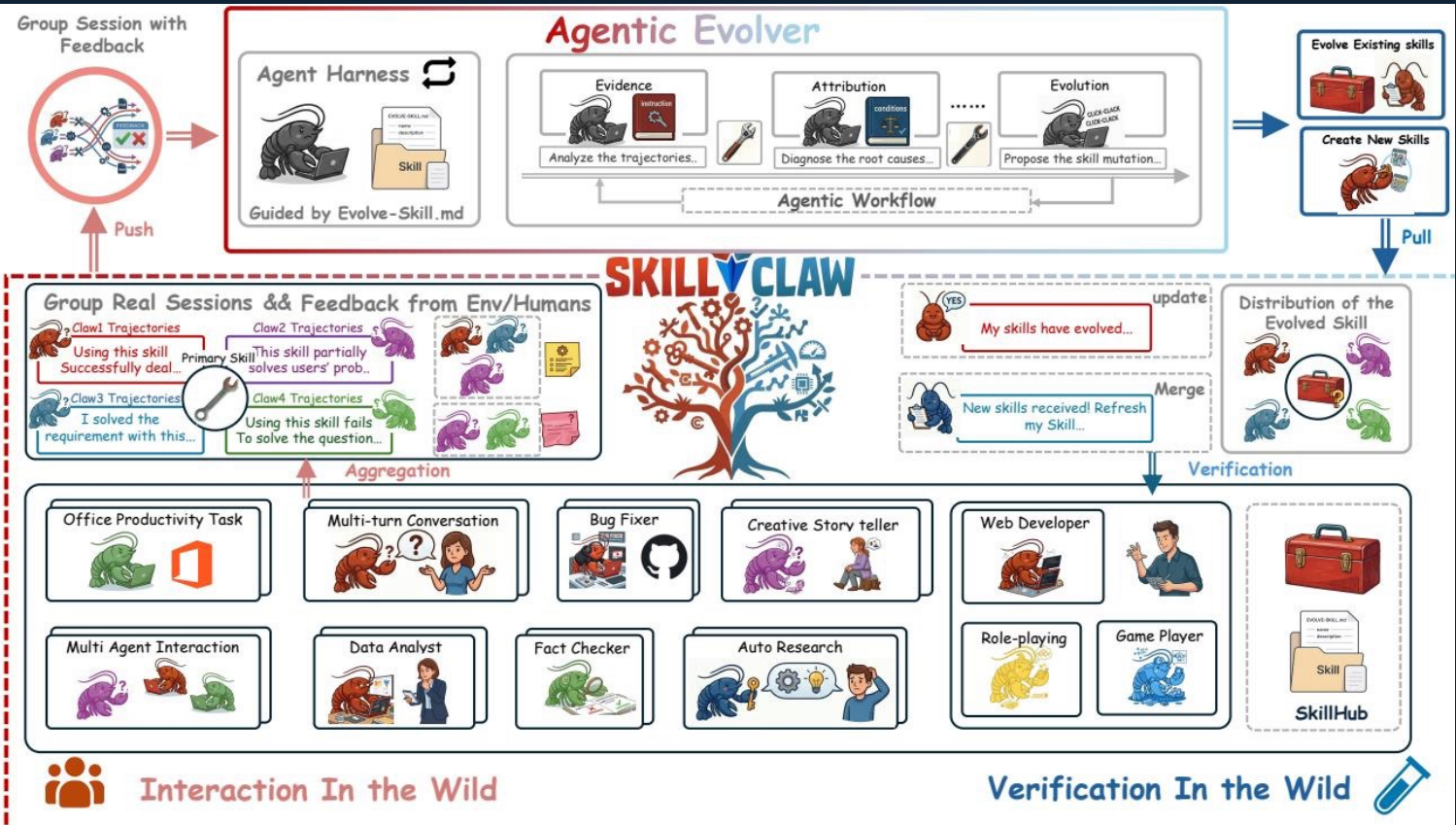
VS

EVOLVING SKILLS (SkillClaw)

- Skills evolve from collective interaction
- Cross-user trajectories drive updates
- Shared evidence base per skill
- Agentic evolver proposes mutations
- Verified updates merged into shared repo
- Knowledge accumulates continuously
- System improves with each deployment

Closed-Loop Pipeline: Multi-User Interaction → Shared Skill Evolution

Four-stage architecture connecting runtime evidence to shared skill improvements



① EVIDENCE

Aggregate session trajectories across all users per skill

② ATTRIBUTION

Diagnose root causes of failures and successes

③ EVOLUTION

Propose skill mutations via open-ended agentic reasoning

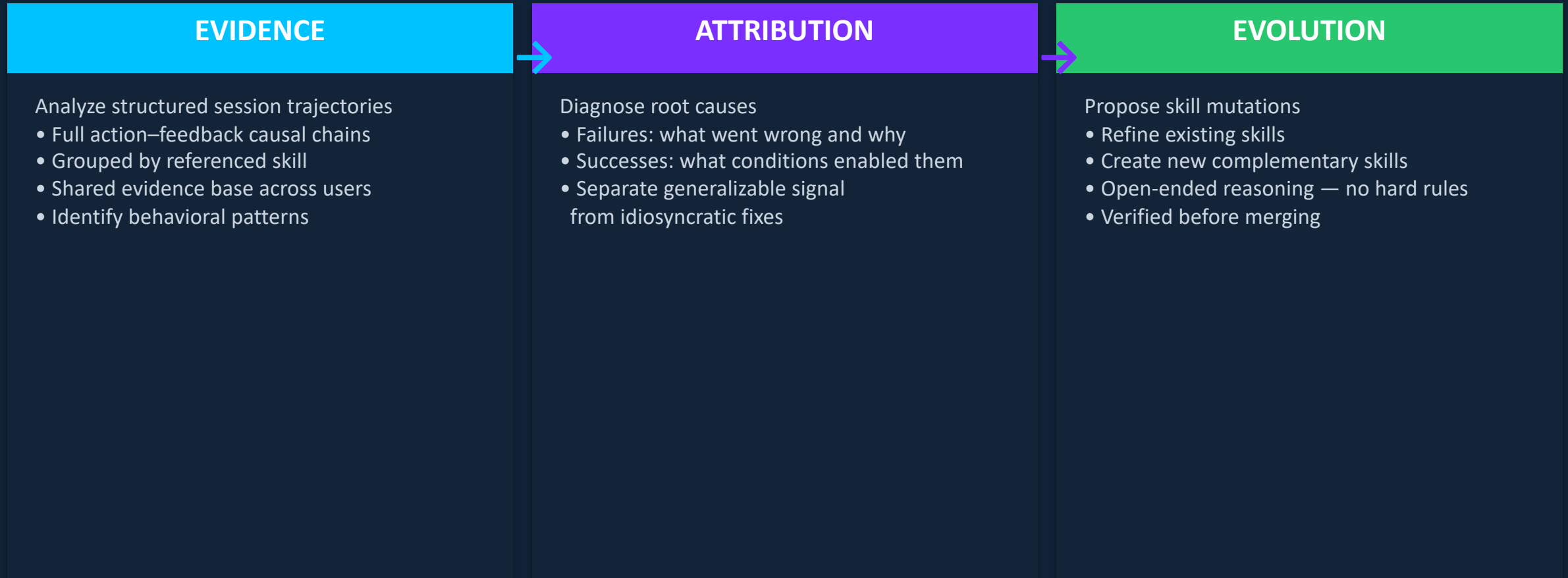
④ DISTRIBUTION

Verify, merge, and sync updated skills to all agents

Multi-user interaction → Session collection → Agentic evolution → Skill synchronization

Agentic Evolver: Evidence → Attribution → Evolution

Three-stage autonomous reasoning pipeline — not predefined rules



The evolver is an autonomous agent performing open-ended reasoning over interaction evidence — not a rule engine. Different users exercising the same skill under diverse contexts produce complementary behavioral signals.

Cross-User Trajectory Aggregation Exposes Skill Behavioral Boundaries

Complementary signals from diverse execution contexts

Compatible Claw-Style Systems

OpenClaw

CoPaw

IronClaw

PicoClaw

ZeroClaw

NanoClaw

NemoClaw

Why Multi-User Aggregation Matters

Different users exercising the same skill under diverse contexts produce complementary views of that skill's behavioral boundary — revealing both conditions under which it works and those under which it breaks.

Single-User Limitation

A single user rarely generates enough signal to separate a generalizable improvement from an idiosyncratic fix. Without aggregation, skill updates risk overfitting to one user's context.

Trajectories preserve full action–feedback causal chains → aggregated per skill → shared evidence base → evolver sees behavioral patterns across the full user population.

Case Study: Multimodal Creative Pipeline Skill — 6 Days of Evolution

Conservative verification: only 1 of 6 candidates accepted

Conservative verification prevents regressions — 1 accepted / 5 rejected across 6 days

Day	Candidate Skill	Skill Function	Verdict	Next-Day Action
1	validate-tmp-workspace-inputs	Check /tmp_workspace inputs & environment setup	Accept	Upgrade to new best pool
2	multimodal-input-validation-and-setup	Multimodal input validation & output env init	Reject	Keep current best pool
3	multimodal-creative-task-pipeline	Multimodal creative pipeline (PDF/video/image)	Reject	Keep current best pool
4	multimodal-creative-task-pipeline (impr.)	Added image classification, visual generation, structured validation	Reject	Keep current best pool
5	...pipeline (impr.) + validate-required-input-files	Creative pipeline + per-file fail-fast validation	Reject	Keep current best pool
6	multimodal-creative-task-pipeline (cand.)	Extended PDF-to-poster / document-to-visual paths	Reject	Not admitted to next cycle

Case Study: Git Push Auth-Fallback Skill — Iterative Refinement Over 6 Days

3 of 6 candidates accepted — iterative improvement before plateau

Iterative refinement: 3 accepted → plateau reached on days 5–6 (3 accepted / 2 rejected / 1 retest)

Day	Candidate Skill	Change Summary	Verdict	Next-Day Action
1	git-push-with-auth-fallback	Safe fallback instead of blocking on push failure	Accept	Add to Safety best pool
2	git-push-with-auth-fallback	Unified patch/bundle filenames, reduced inconsistency	Accept	Keep updated best pool
3	git-push-with-auth-fallback + git-clone-to-directory	Auth-alternative paths & secrets audit; fixed subshell pitfalls	Accept	Keep current best pool
4	(none — retest)	Same-pool retest; validator confirmed current pool as best	Accept	Continue same best pool
5	git-push-with-auth-fallback	Added 'push hang treated as auth failure'; no improvement	Reject	Keep current best pool
6	git-push-with-auth-fallback	Added identity config & filename consistency; no improvement	Reject	Not admitted to next cycle

Contrast with creative pipeline (1/6): when evidence strongly supports a skill direction, SkillClaw refines iteratively. Verification halts further updates once improvements plateau.

Three Properties That Distinguish SkillClaw

Implications for continuously improving agent systems

① Collective Evolution

Individual interactions feed a shared, continuously improving skill ecosystem. Knowledge no longer stays locked inside a single user session — every execution contributes to the collective intelligence of the system.

② Fully Automatic

Skill evolution is driven entirely by runtime interaction without manual curation or explicit user intervention. No human-in-the-loop required for the evolution cycle.

③ Agentic Evolution Paradigm

Skill updates are produced through open-ended agent reasoning over evidence — not by predefined update rules or hard-coded heuristics. The evolver adapts to whatever patterns the trajectories reveal.

Implication: Building on SkillClaw, agent ecosystems can achieve compound improvement — each deployment round produces a better skill library, evaluated on WildClawBench with Qwen3-Max across OpenClaw, CoPaw, IronClaw, PicoClaw, ZeroClaw, NanoClaw, and NemoClaw.